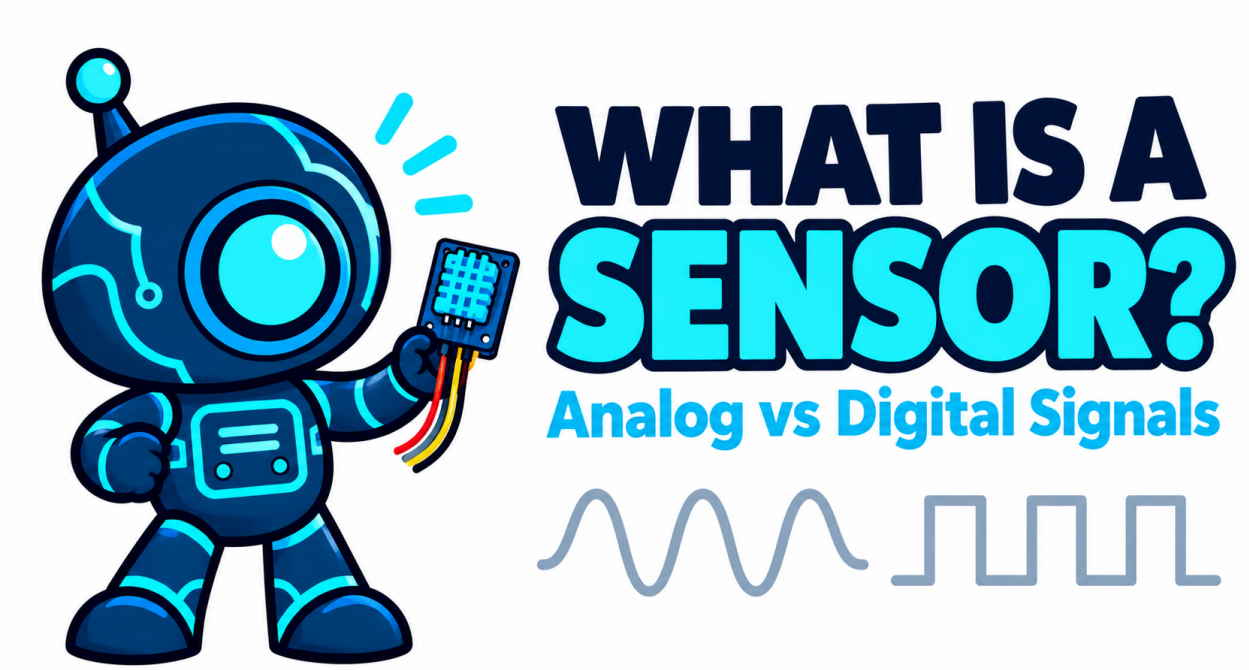
AI MECH LAB

Mechatronics Course — Beginner Stage

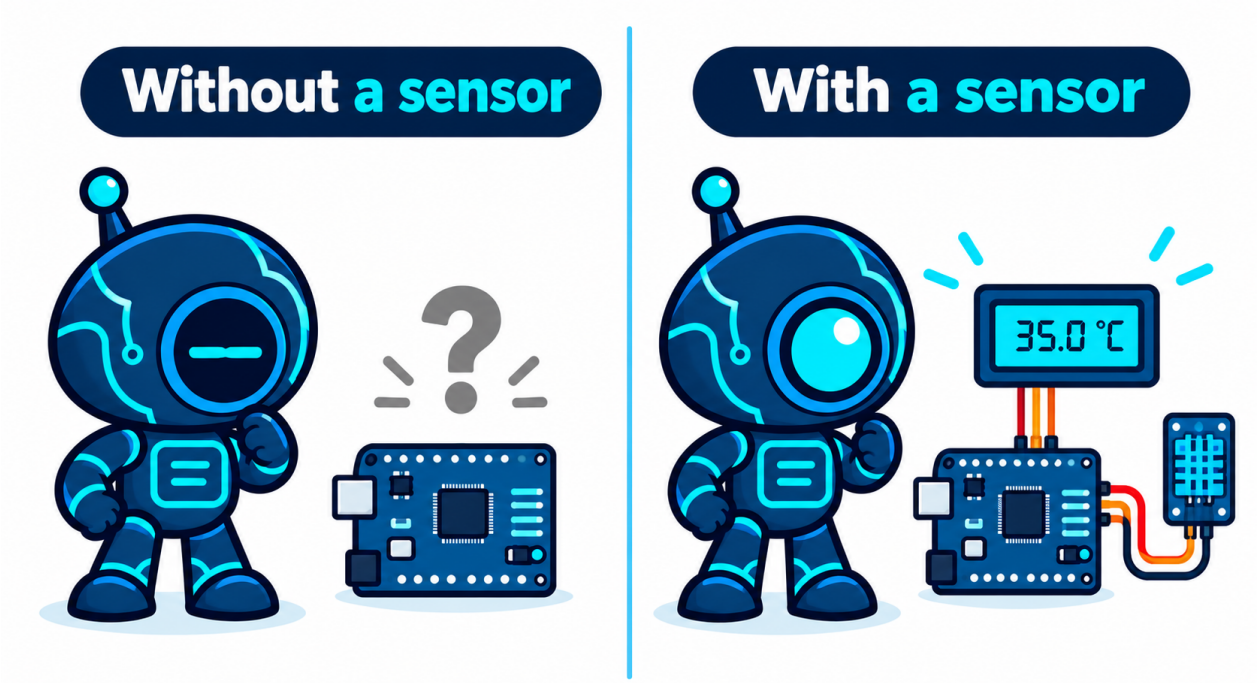


Module 3 · Topic 3.1

What Is a Sensor? Analog vs. Digital Signals

What Is a Sensor? Analog vs. Digital Signals

A microcontroller is excellent at arithmetic and completely blind to the world. It has no idea whether an oven is at 20 °C or 200 °C, whether a robot arm has reached the end of its travel, or whether a generator shaft is shaking itself apart. Everything it will ever know about the physical world arrives as an electrical signal, and something has to produce that signal. That something is a sensor.

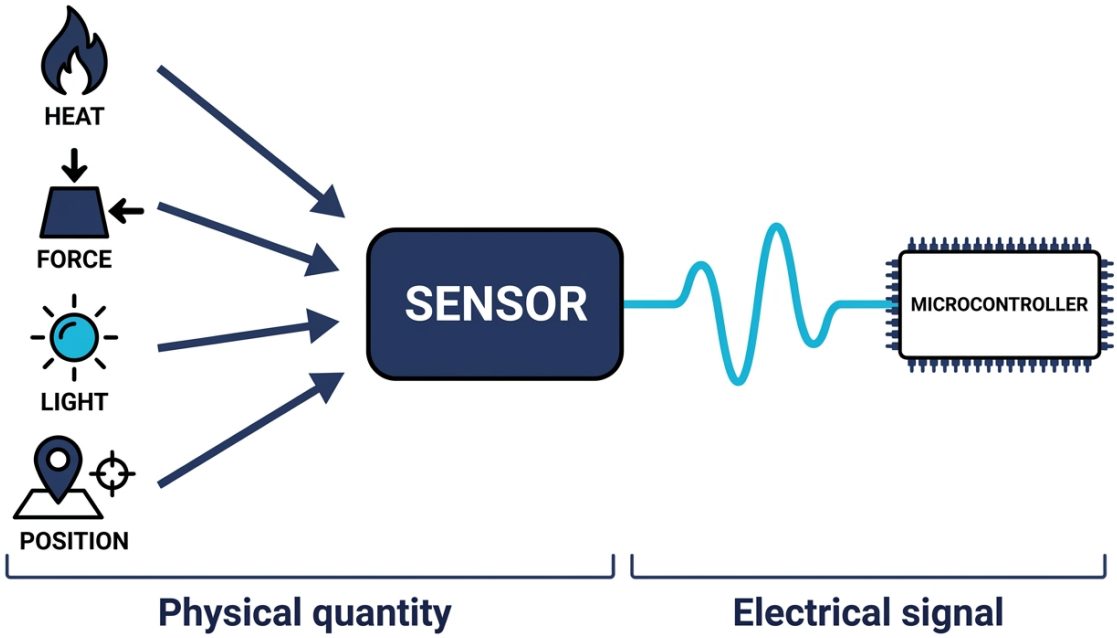


Without a sensor the controller is blind. With one, it knows what the physical world is doing.

What a sensor is

A sensor is a device that converts a physical quantity into an electrical signal that a circuit can read. The quantity might be temperature, force, light, position, speed, pressure or vibration. The signal is usually a voltage, though it can also be a current, a resistance, or a stream of digital data.

Inside most sensors, a sensing element changes some electrical property when the physical quantity changes. A thermistor changes its resistance as it heats up. A strain gauge changes its resistance as it stretches. A photodiode changes its output as light falls on it. A small circuit then turns that change into something your controller can use, most often a voltage. You will also hear the word transducer, which means any device that converts one form of energy into another. A sensor is a transducer whose output is electrical.

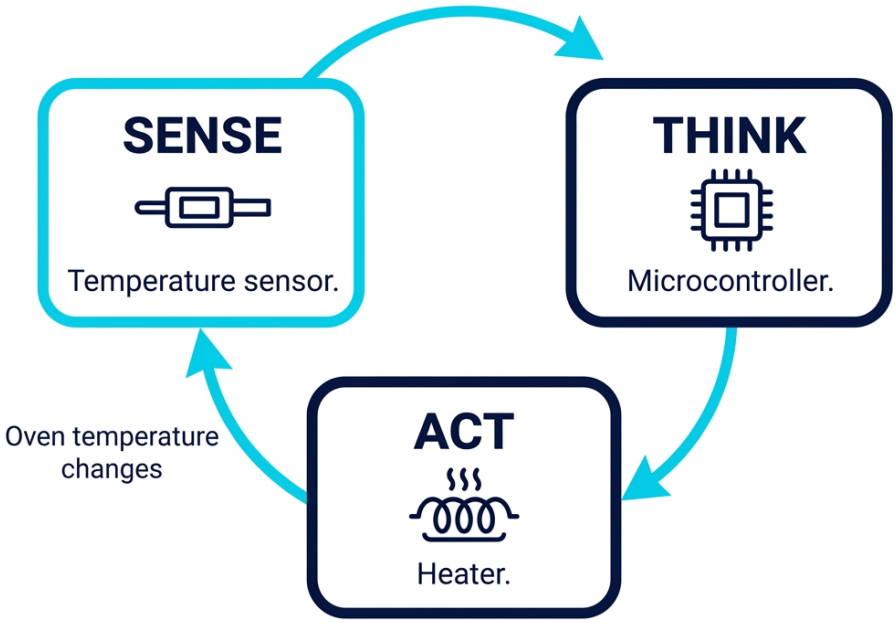


A sensor translates a physical quantity into an electrical signal the controller can read.

This has a consequence worth holding onto. A sensor never hands you a temperature. It hands you a voltage that you convert into a temperature using what the datasheet tells you about the relationship. Every sensor reading is an electrical measurement that your software interprets.

Where sensors sit in a mechatronic system

Every mechatronic system follows the same loop: sense, think, act. Take an industrial oven. A temperature sensor reports the current temperature, the microcontroller compares it to the target, and a heater switches on or off to close the gap. The sensor is the first link. Without it, the controller is guessing, and a controller that guesses is just a timer. Module 4 covers the actuators at the other end of the loop. This module is about the sensing end.

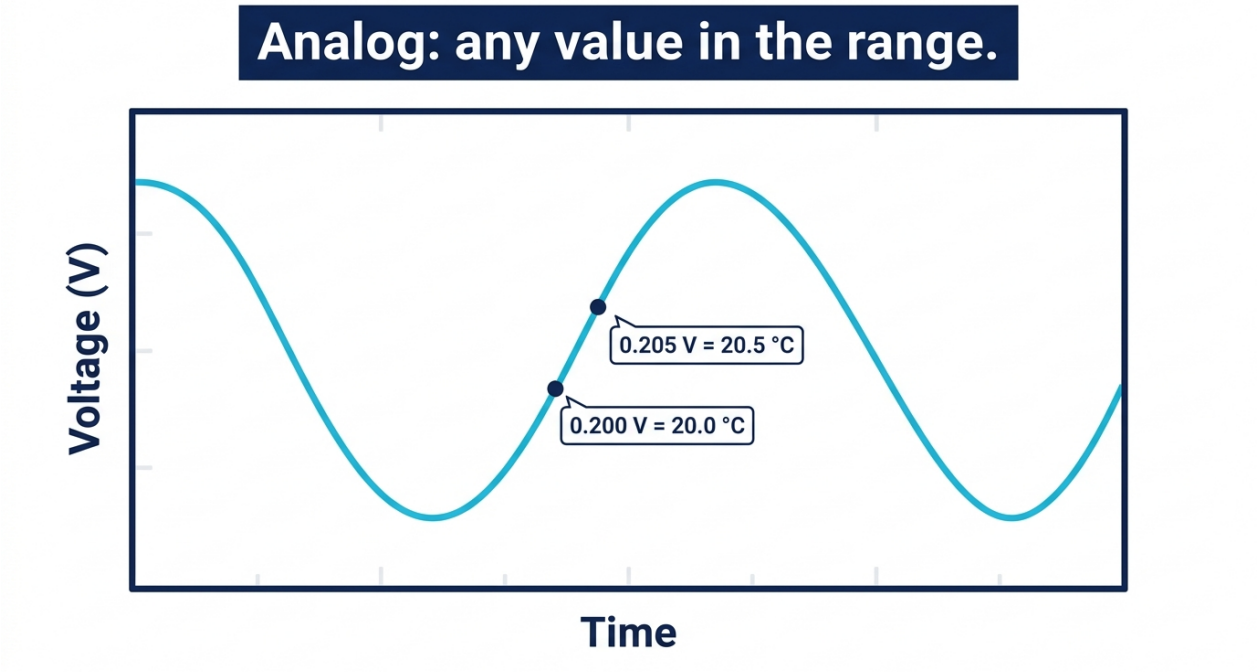


The sense-think-act loop in an oven. This module focuses on the first link, sensing.

Two kinds of signal

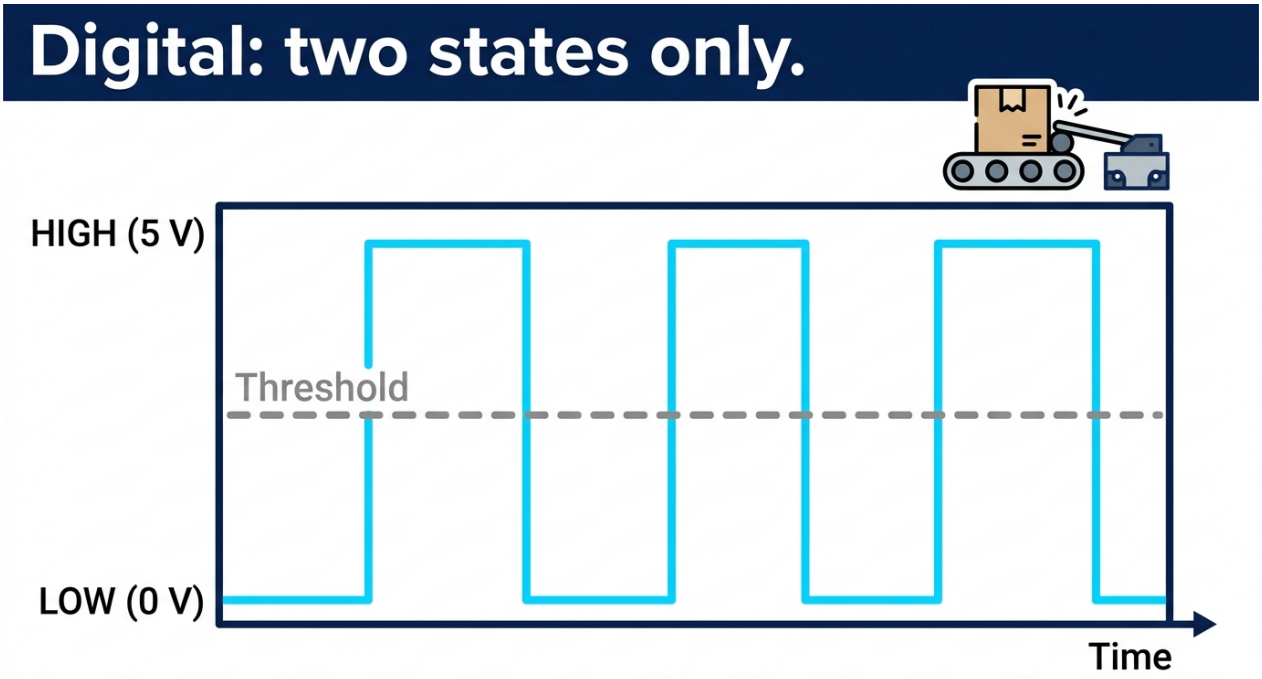
A signal is a quantity that carries information as it changes over time. Sensors produce two kinds, and the difference decides how you wire them, read them and trust them.

Analog signals can take any value within a range, and they vary smoothly. Consider a temperature sensor that outputs about 10 millivolts for every degree Celsius, which is how some common sensors work. At 20 °C it outputs 0.200 V. At 20.5 °C it outputs 0.205 V. At 20.51 °C it outputs 0.2051 V. There is no gap between possible values. A potentiometer attached to a robot joint behaves the same way: its output voltage follows the joint angle continuously. Analog signals carry rich information, and the sensors producing them are often simple. Their weakness is noise. Small unwanted voltage variations picked up along the wire add error that you cannot easily separate from the real signal, and a long wire running through a noisy factory makes it worse.



An analog signal can take any value in its range. The 10 mV per degree figure is illustrative.

Digital signals take only two values: high or low, on or off, 1 or 0. On a typical microcontroller, high is close to the supply voltage, commonly 3.3 V or 5 V, and low is close to 0 V. A limit switch at the end of a conveyor is a good example: either the part has arrived and pressed it, or it has not. Digital signals are robust, because noise has to be large enough to push the voltage across the threshold between low and high before it causes an error. The cost is information. One wire carries one yes-or-no answer.



A digital signal has two states. Noise must cross the threshold to cause an error.

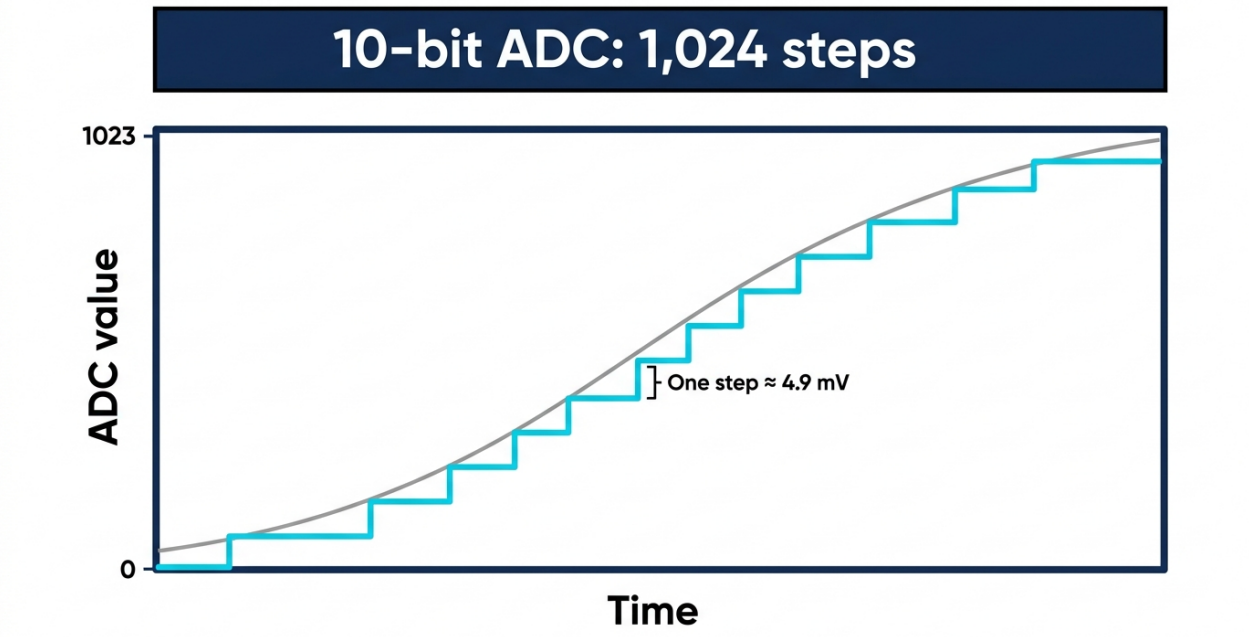
The word digital also covers a second kind of sensor. Many modern sensors contain their own small electronics that measure, convert and then send the result as a number, as a stream of high and low pulses following an agreed communication protocol. You will use these in Topic 3.3. The distinction to keep clear is between a digital on/off signal, which is a single two-state wire, and a digital data sensor, which sends actual measurements as numbers.

Turning analog into numbers: the ADC

A microcontroller's processor works with numbers, so an analog voltage has to be converted before your code can use it. The circuit that does this is the analog-to-digital converter, or ADC. It measures the voltage at one instant and reports it as a whole number. Two figures define how well it does that.

Resolution is the number of bits the ADC uses. An n-bit ADC can report 2 to the power n different values. A 10-bit ADC reports 1,024 levels, numbered 0 to 1,023. If it spans 0 to 5 V, each step is about 4.9 mV. Any change smaller than one step is invisible to it.

Sampling rate is how often the ADC takes a reading. The faster the quantity changes, the faster you must sample. A slow oven needs a reading every second. A vibrating generator shaft needs thousands per second.



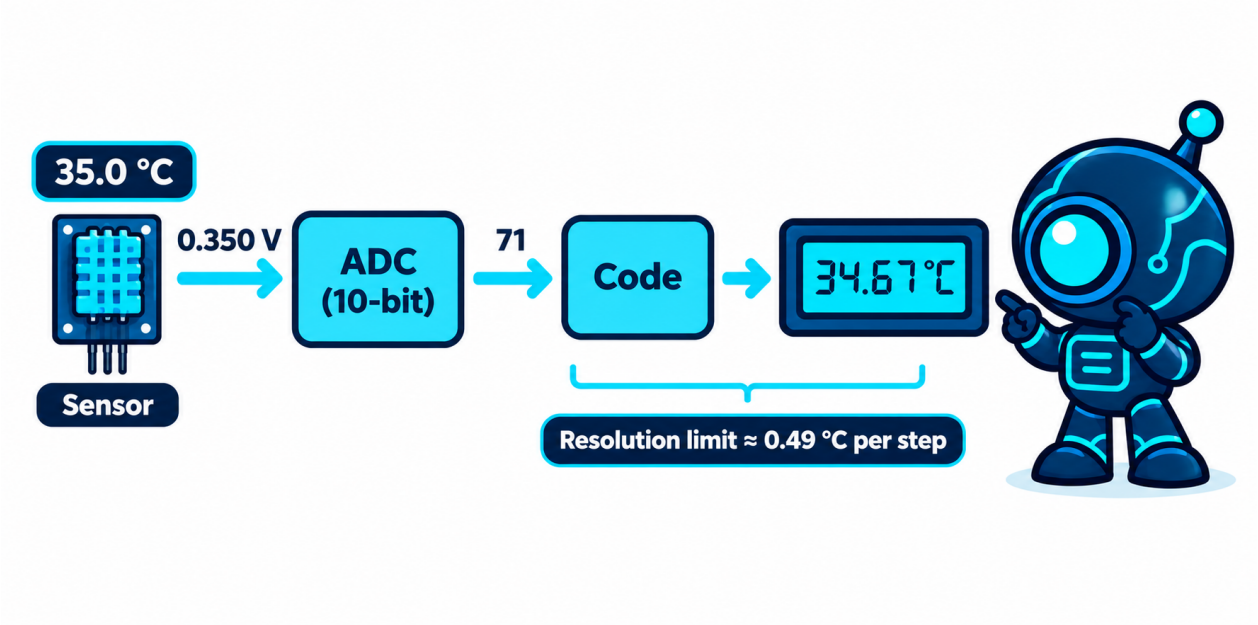
A 10-bit ADC turns a smooth signal into 1,024 discrete steps.

A worked example

Suppose the temperature sensor above, at 10 mV per °C, feeds a 10-bit ADC with a 5 V range. The room is exactly 35.0 °C, so the sensor outputs 0.350 V. The ADC computes 0.350 divided by 5, multiplied by 1,024, which is 71.68. It reports whole numbers, so your code receives 71.

To convert back, your code multiplies 71 by 5/1,024, which gives 0.3467 V, and divides by 10 mV per °C. The result is 34.67 °C. That is not a fault. It is the resolution limit at work. Each ADC step

here equals about 0.49 °C, so this setup cannot tell 35.0 °C from 34.8 °C. If your application needs to see changes of 0.1 °C, you need a higher-resolution ADC, or you amplify the sensor signal before it reaches the ADC so each degree covers more steps.



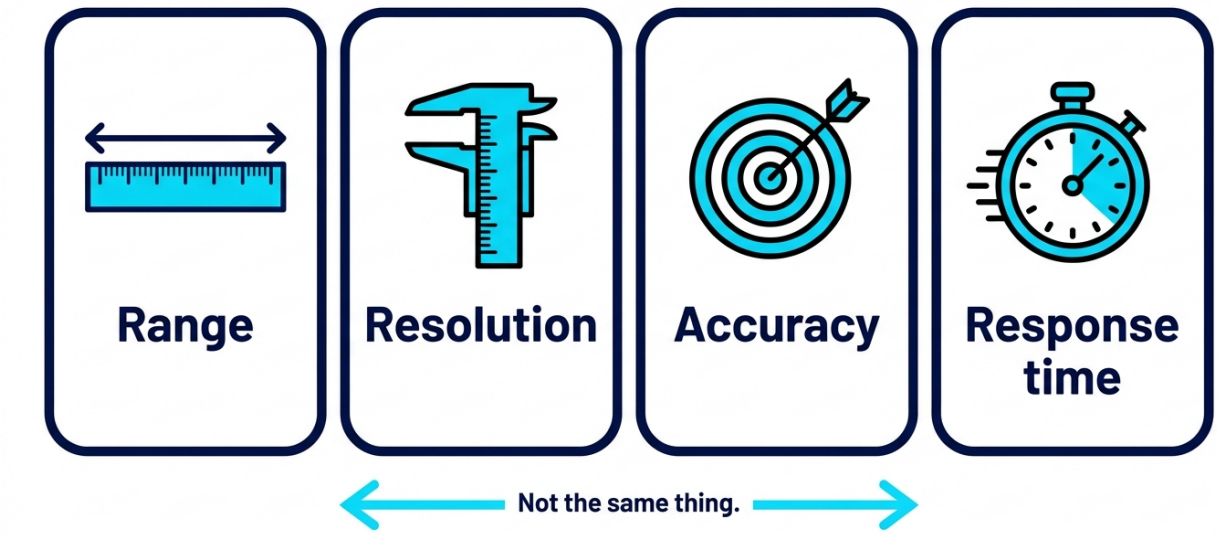
From sensor to ADC to code: the reading lands on 34.67 °C because of the resolution limit.

Choosing between them

Use an analog sensor when you need a continuous measurement, such as temperature, position or pressure, and your controller has an ADC. Use a digital on/off sensor when the question is a plain yes or no: is the door closed, did the part arrive, has the arm reached its limit. Use a digital data sensor when you want precision and noise immunity and your controller supports its protocol.

What to check before you pick a sensor

Four specifications matter on every sensor datasheet, which is where the skill from Topic 2.4 pays off. **Range** is the span the sensor can measure. **Resolution** is the smallest change it can show. **Accuracy** is how close its reading is to the true value. **Response time** is how quickly the output follows a change. Resolution and accuracy are not the same thing. A sensor can display values in steps of 0.01 degrees and still be wrong by 2 degrees.



Four specifications to check on every sensor datasheet.

> ***Going deeper:*** *the Robotics course revisits sensors from the point of view of machines that move through the world and combine several readings at once. Its module on sensors in robotics builds on everything in this topic.*

Where this leaves you

You now know what a sensor really does: it converts a physical quantity into an electrical signal, and your software turns that signal back into meaning. You can tell analog signals from digital ones, you know why an ADC limits how finely you can read an analog sensor, and you have four specifications to check before choosing one. Next comes Topic 3.2, where we survey the common sensor types you will actually meet on a bench, from temperature and light sensors to distance and motion sensors, and what each one outputs.